

RAZONAMIENTO AUTOMÁTICO PROYECTO 2: MACHINE LEARNING

Miguel Martínez Celdrán
Jordi Calatayud Álvarez
Maria Victoria Lanzeni Llop

Índice

Perceptrón	3
Estructura y diseño del código	3
Dataset 1 - Heart_Diseases	4
Dataset 2 - Iris	6
Dataset 3 - Cáncer	7
Red Neuronal con BackPropagation	9
Estructura básica	9
Funcionamiento	10
Inicialización	10
Forward	10
Backpropagation	12
Funcionalidad	14
Mejoras implementadas y mejoras posibles	14
Datasets para red	15
Indicador de salud según uso de redes	15
Pruebas y ajustes de hiperparámetros	18
Tipos de flores iris	19
Pruebas y ajustes de hiperparámetros	22
Heart disease	23
Pruebas y ajustes de hiperparámetros	23
Conclusiones	26
Red Neuronal con pesos entrenados con Algoritmos Genéticos	26
Idea básica	26
Implementación	26
Mejoras de la implementación	27
Función fitness	27
Mutaciones	27
Probando la red con distintos datasets	28
Heart Disease Prediction	28
Pruebas y ajustes de hiperparametros	28
Tipos de flores iris	31
Pruebas y ajustes de hiperparametros	32
Indicador de salud según uso de redes	33
Pruebas y ajustes de hiperparametros	33
Conclusiones	34
Bibliografía	34

Perceptrón

Estructura y diseño del código

En este apartado se diseña un perceptrón binario capaz de resolver problemas de clasificación binarios y se amplía a multiclase mediante una estrategia One-vs-Rest, entrenando un perceptrón binario por cada clase. El modelo aprende de un vector de pesos 'w' y un bias 'b', a partir de ejemplos (x, y) donde :

- x : vector numérico de características de dimensión d.
- y (0, 1) : etiqueta binaria . 0 = clase negativa, 1= clase positiva.

En multiclase, las etiquetas originales se mapean a 0...k-1, y para cada clase se construye un problema binario 1-vs-Rest. La predicción del perceptrón se obtiene aplicando un umbral sobre la suma ponderada:

$$s = \sum_{j=0}^{d-1} w_j x_j + b \quad \begin{array}{l} \rightarrow y = 1 \text{ si } s \geq 0 \\ \rightarrow y = 0 \text{ si } s < 0 \end{array}$$

Para conseguir esto, en el código del perceptrón primero cargamos el dataset correspondiente con "csv.DictReader", lo cual permite acceder a cada columna por su nombre. Antes de entrenar, se normalizan todas las columnas de entrada al rango [0, 1] mediante normalización Min-Max aplicada al vector final de características. La partición en entrenamiento y en test es 80-20 (train-test) y se mezclan de manera aleatoria con una semilla, para que el split sea reproducible. Luego, la función "train_perceptron" se entrena usando la regla clásica de actualización cuando hay error de clasificación. Los parámetros principales son :

- epochs : número de pasadas completas por el conjunto de entrenamiento
- lr : learning rate, controla el tamaño del ajuste de pesos.
- seed : semilla para barajar ejemplos en cada época

En cuanto al vector de pesos y bias :

- d = len(X[0]) es el número de features
- w = [0..d-1] son los pesos asociados a cada feature
- w [d] es el bias

Luego la predicción se obtiene aplicando el umbral "pred =1 if s >= 0 else 0".

Se define el error como la diferencia obtenida y luego si el error es distinto de 0, para cada peso (y bias) :

- w[j] += lr * err * xi[j]
- w[d] += lr * err

En cada época se baraja el orden de ejemplos evitando así dependencia del orden del dataset y favoreciendo un mejor aprendizaje.

Una vez entrenado el modelo se evalúa sobre el conjunto de test, construimos los cuatro conteos básicos (TP, FP, TN, FN) y medimos la Accuracy, la Precision y el Recal. En el caso multiclase, se utiliza una matriz de confusión y se calculan las métricas Precision, Recall y F1 en su versión macro, promediando el resultado entre clases.

En cuanto a la estrategia One-vs-Rest utilizada en la multiclase, cada perceptrón compete dando una puntuación (score). La predicción final es la clase cuyo perceptrón obtiene el score más alto.

El código está diseñado para ser reutilizable con distintos datasets sin modificar el código gracias a los argumentos de línea de comandos. A continuación los nombro y pongo un ejemplo de uso junto a su explicación :

- - - **csv datasets/iris.csv** : Indica qué conjunto de datos se va a cargar y procesar
- - - **target Species** : Le dice al programa qué columna debe usarse como salida “y” del perceptrón
- - - **classes “Iris-setosa,Iris-versicolor,Iris-virginica”** : Lista de todas las clases posibles, separadas por comas.
- - - **drop Id** : (OPCIONAL) Columnas que se eliminan completamente del dataset (porque son irrelevantes, por ejemplo)
- - - **date_cols date** : (OPCIONAL) Columnas que contienen fechas (para poder transformarlas)
- - - **cat_cols gender,platform** : (OPCIONAL) Columnas categóricas (texto con valores discretos). El loadore asume que las columnas no marcadas como categóricas o fecha son numéricas
- - - **epochs 20** : (OPCIONAL) Número de epochs del conjunto de entrenamiento.
- - - **lr 0.1** : (OPCIONAL) Learning rate
- - - **test_ratio 0.2** : (OPCIONAL) Porcentaje del dataset que se reserva para el test
- - - **seed 0** : (OPCIONAL) Semilla del generador aleatorio

El uso de estos argumentos por línea de comandos nos permite reutilizar el mismo código con distintos datasets y configuraciones experimentales sin necesidad de modificar el script, facilitando la experimentación y la reproducibilidad.

Dataset 1 - Heart_Diseases

El dataset utilizado para estas primeras pruebas es el siguiente:

<https://www.kaggle.com/datasets/neurociper/heartdisease?resource=download>

La variable objetivo es Heart Disease, con clase positiva Presence y clase negativa Absence. El dataset contiene 270 muestras y 13 variables predictoras numéricas (edad, presión arteria, colesterol...). La distribución de clases está balanceada (120 positivos - 150 negativos), por lo que pensamos que puede ser adecuado entrenarlo con este dataset.

Para la primera evaluación hemos establecido como configuración base 20 epochs y un learning rate de 0.1. Estos han sido los resultados:

```
[cpc@CPCtelera heart_disease]$ cat baseline_e20_lr0.1_seed0.txt
Dataset: datasets/Heart_Disease_Prediction.csv
Samples: 270
Features: 13 -> ['Age', 'Sex', 'Chest pain type', 'BP', 'Cholesterol', 'FBS over 120', 'EKG results', 'Max HR', 'Exercise angina', 'ST depression', 'Slope of ST',
'Number of vessels fluoro', 'Thallium']
Class balance: y=1 -> 120, y=0 -> 150
Split: train=216, test=54

Resultados en TEST:
epochs=20, lr=0.1
Accuracy : 0.7778
Precision: 0.6538
Recall   : 0.8500
(TP, FP, TN, FN) = (17, 9, 25, 3)

Matriz de confusión (Pred vs Real):
              Real=1  Real=0
Pred=1 (Pos)    17     9
Pred=0 (Neg)     3    25
[cpc@CPCtelera heart_disease]$
```

En la configuración base hemos obtenido un accuracy del 77.8%. El recall (0.85) es elevado, lo que indica que el perceptrón identifica correctamente la mayoría de pacientes

con presencia de enfermedad (solo hay 3 FN). Sin embargo, la precisión (0.65) es menor debido al número de falsos positivos (9).

Para realizar unas pruebas más exhaustivas voy a realizar varios experimentos más, variando epochs manteniendo el learning rate , variando el learning rate manteniendo 20 epochs y variando la semilla para mostrar estabilidad.

He comenzado variando las epochs y dejando el learning rate y la semilla iguales, y he obtenido los siguientes resultados:

- **e1_lr01_seed0**: acc 0.7963, prec 0.6957, rec 0.8000
- **e10_lr01_seed0**: acc 0.8333, prec 0.7619, rec 0.8000
- **e20_lr01_seed0 (baseline)**: acc 0.7778, prec 0.6538, rec 0.8500
- **e50_lr01_seed0**: acc 0.8519, prec 0.8000, rec 0.8000
- **e100_lr01_seed0**: acc 0.7593, prec 0.6400, rec 0.8000

En estas pruebas 50 epochs ofrece el mejor equilibrio, mientras que se ve que al aumentar las epochs empeora el rendimiento, lo que sugiere que el perceptrón podría oscilar y no converger a una solución estable para este split.

También se han probado variaciones del Learning rate manteniendo 20 epochs y la misma semilla y en todas las variaciones (0.01, 0.05, 0.1, 0.2, 0.5 y 1.0) hemos obtenido exactamente acc 7778, prec 0.6538, rec 0.8500 . Esto indica que el perceptrón converge muy rápido y todos los learning rates probados están en un rango seguro. En este contexto el learning rate no es un hiperparámetro crítico, a diferencia del número de epochs o el particionado del dataset.

Por último se ha probado un cambio con la semilla, manteniendo 20 epochs y learning rate 0.1 y probando como semilla 1, 2, 3. Estos han sido los resultados:

- **e20_lr01_seed0**: acc 0.7778, prec 0.6538, rec 0.8500
- **e20_lr01_seed1**: acc 0.6111, prec 1.0000, rec 0.1600
- **e20_lr01_seed2**: acc 0.8148, prec 0.7647, rec 0.9286

Recordemos que la semilla cambia el barajado de muestras y el split train/test, y eso puede afectar bastante al perceptrón. Vemos que con semilla 1 el modelo se vuelve muy conservador (con precisión de 1.0), solo predice positivo cuando está muy seguro, lo cual cuadra con el bajo recall que significaría que se le escapan muchos positivos.

Con semilla 2 el rendimiento mejora y el modelo detecta más positivos y con buena precisión.

Dataset 2 - Iris

El dataset utilizado para estas segundas pruebas es el siguiente :

<https://www.kaggle.com/datasets/uciml/iris>

Hemos realizado un primer experimento base con este dataset de 150 muestras bien balanceadas donde el objetivo es clasificar flores en 3 especies a partir de 4 medidas numéricas de la flor.

Para conseguir que el perceptrón inicial sea capaz de clasificar en 3 clases hemos usado una estrategia donde en vez de entrenar un único perceptrón, entrenamos un perceptrón por clase. De esta manera cada perceptrón aprende a responder a una pregunta tipo "¿Es Iris-setosa o NO?". Cuando vamos a clasificar una flor nueva, la pasamos por los 3

perceptrones y cada uno da un score y nos quedamos con la clase del perceptrón que haya dado el score más alto, así , aunque el perceptrón sea binario por separado, en conjunto podemos evaluar qué tal funciona en una clasificación ternaria.

Este dataset tiene 4 columnas numéricas asíque con normalizar las 4 medidas con Min-Max para dejarlas entre 0-1 es suficiente para ayudar al resultado. Dividimos el dataset en 80-20 (train-test).

El primer experimento baseline que usamos consistió en 20 epochs, un learning rate de 0.1 y una semilla de 0. Los resultados de este test han sido

```
Resultados en TEST :
epochs=20, lr=0.1
Accuracy      : 0.7667
Precision(M)  : 0.8772
Recall(M)     : 0.7407
F1(M)        : 0.7126

Matriz de confusión (Pred vs Real):
                Real=Iris-setosa  Real=Iris-versicolor  Real=Iris-virginica
Pred=Iris-setosa          9           0           0
Pred=Iris-versicolor      0          12           7
Pred=Iris-virginica       0           0           2
[cpc@CPCtelera iris]$
```

La Setosa sale perfectamente predicha, en la Versicolor hay 12 correctas pero 7 virginica se predicen como versicolor, solo 2 virginicas son detectadas, 7 se van a versicolor. Podemos apreciar según esta prueba que Setosa es linialmente separable y fácil, y el problema lo tenemos en separar versicolor y virginica, que suelen solaparse y el perceptrón se queda corto. Aun así , obtenemos un 0.766 de accuracy.

De nuevo, realizando pruebas variadas he probado a variar las epochs, el learning rate y la semilla. En el primer conjunto de pruebas hemos variado el número de epochs manteniendo constantes el learning rate a 0.1 y la semilla a 0:

- **e1_lr01_seed0:**
Accuracy = 0.6000, Precision(M) = 0.4762, Recall(M) = 0.6667, F1(M) = 0.5333
- **e5_lr01_seed0:**
Accuracy = 0.7333, Precision(M) = 0.7741, Recall(M) = 0.7685, F1(M) = 0.7315
- **e10_lr01_seed0:**
Accuracy = 0.8667, Precision(M) = 0.9167, Recall(M) = 0.8519, F1(M) = 0.8571
- **e20_lr01_seed0 (baseline):**
Accuracy = 0.7667, Precision(M) = 0.8772, Recall(M) = 0.7407, F1(M) = 0.7126
- **e50_lr01_seed0:**
Accuracy = 0.6000, Precision(M) = 0.4000, Recall(M) = 0.6667, F1(M) = 0.5000
- **e100_lr01_seed0:**
Accuracy = 0.9667, Precision(M) = 0.9744, Recall(M) = 0.9630, F1(M) = 0.9671

Observamos que con muy pocas epochs el modelo no aprende suficiente, y con 100 epochs ya obtiene unos muy buenos resultados. La caída del accuracy con 50 epochs puede deberse al orden de las muestras con las que se ha entrenado.

Si medimos cómo afecta el learning rate nos damos cuenta de que siempre se obtienen los mismos resultados :

Accuracy = 0.7667 Precision(M) = 0.8772 Recall(M) = 0.7407 F1(M) = 0.7126

Esto indica que de nuevo, el perceptrón converge muy rápidamente y todos los valores del learning rate probados se encuentran en un rango estable.

Por último, hemos analizado la influencia de la semilla aleatoria manteniendo 20 epochs y learning rate en 0.1. Los resultados han sido los siguientes :

- **seed = 0:**
Accuracy = 0.7667, F1(M) = 0.7126
- **seed = 1:**
Accuracy = 1.0000, Precision(M) = 1.0000, Recall(M) = 1.0000, F1(M) = 1.0000
- **seed = 2:**
Accuracy = 0.9667, F1(M) = 0.9696
- **seed = 3:**
Accuracy = 0.9667, F1(M) = 0.9663
- **seed = 4:**
Accuracy = 0.6000, F1(M) = 0.4934

En este caso la semilla tiene un impacto muy importante en el rendimiento del perceptrón, debido a que afecta particionando el train/test y en el orden en el que se presentan las muestras durante el entrenamiento.

Con estas pruebas vemos que el perceptrón es un modelo lineal sencillo que funciona bien en problemas relativamente separables, pero que puede mostrar inestabilidad dependiendo del orden de entrenamiento y de la distribución de los datos.

Dataset 3 - Cáncer

Para el tercer experimento utilizamos un dataset médico orientado a estimar el riesgo de cáncer de pulmón a partir de información como hábitos , variables clínicas...

<https://www.kaggle.com/dhrubangtalukdar/lung-cancer-prediction-dataset>

El objetivo es predecir la columna lung_cancer_risk, que es una etiqueta binaria donde 0 es riesgo bajo o ausencia de riesgo y 1 es riesgo alto. Hay 29 características numéricas y 5000 muestras con un reparto algo desbalanceado ya que hay 3756 muestras de la clase 0 y 1244 de la clase 1.

A continuación se realiza una primera ejecución baseline con 20 epochs, learning rate 0.1 y seed 0, usando un split 80/20 (train-test). Los resultados obtenidos son los siguientes :

```

[cpc@CPCtelera lung_cancer]$ cat baseline_e20_lr01_seed0.txt
Dataset: datasets/lung_cancer.csv
Samples: 5000
Features: 29
Target: lung_cancer_risk -> ['0', '1']
Class balance:
  0 -> 3756
  1 -> 1244
Split: train=4000, test=1000

Resultados en TEST :
epochs=20, lr=0.1
Accuracy      : 0.9830
Precision(M)  : 0.9873
Recall(M)     : 0.9697
F1(M)         : 0.9780

Matriz de confusión (Pred vs Real):
                Real=      0   Real=      1
Pred=      0           729         16
Pred=      1           1          254
[cpc@CPCtelera lung_cancer]$

```

Hemos obtenido muy buenos resultados esta vez con solo 17 errores en 1000 ejemplos y además casi no confunde la clase 1. Estos valores indican que el perceptrón es capaz de separar muy bien ambas clases. Hemos hecho las mismas pruebas que en los otros 2 datasets anteriores y al variar el número de epochs manteniendo $lr = 0.1$ y la misma semilla se observa que con 1 epoch el rendimiento es algo menor (0.94) y a partir de 10-20 epochs el rendimiento es bastante alto y estable. Con 50 epochs se obtiene el mejor resultado global, con un accuracy de 0.994 y un F1-score de 0.9924. Aumentar hasta 100 epochs no empeora el modelo pero la mejora respecto a 50 epochs es pequeña.

Cuando variamos el learning rate nos pasa igual que en los otros 2 datasets, la accuracy se mantiene en 0.982, la precision en 0.987 y el recall en 0.970, lo que indica que el perceptrón converge a la misma solución independientemente del learning rate dentro de este rango.

Al variar la semilla manteniendo 20 epochs y $lr = 0.1$ se observan ligeras variaciones, aunque el accuracy se mantiene entre 0.979 y 0.991. El F1-score siempre está por encima de 0.97. Esto demuestra que el modelo es estable y no depende excesivamente de una partición concreta en este dataset.

Red Neuronal con BackPropagation

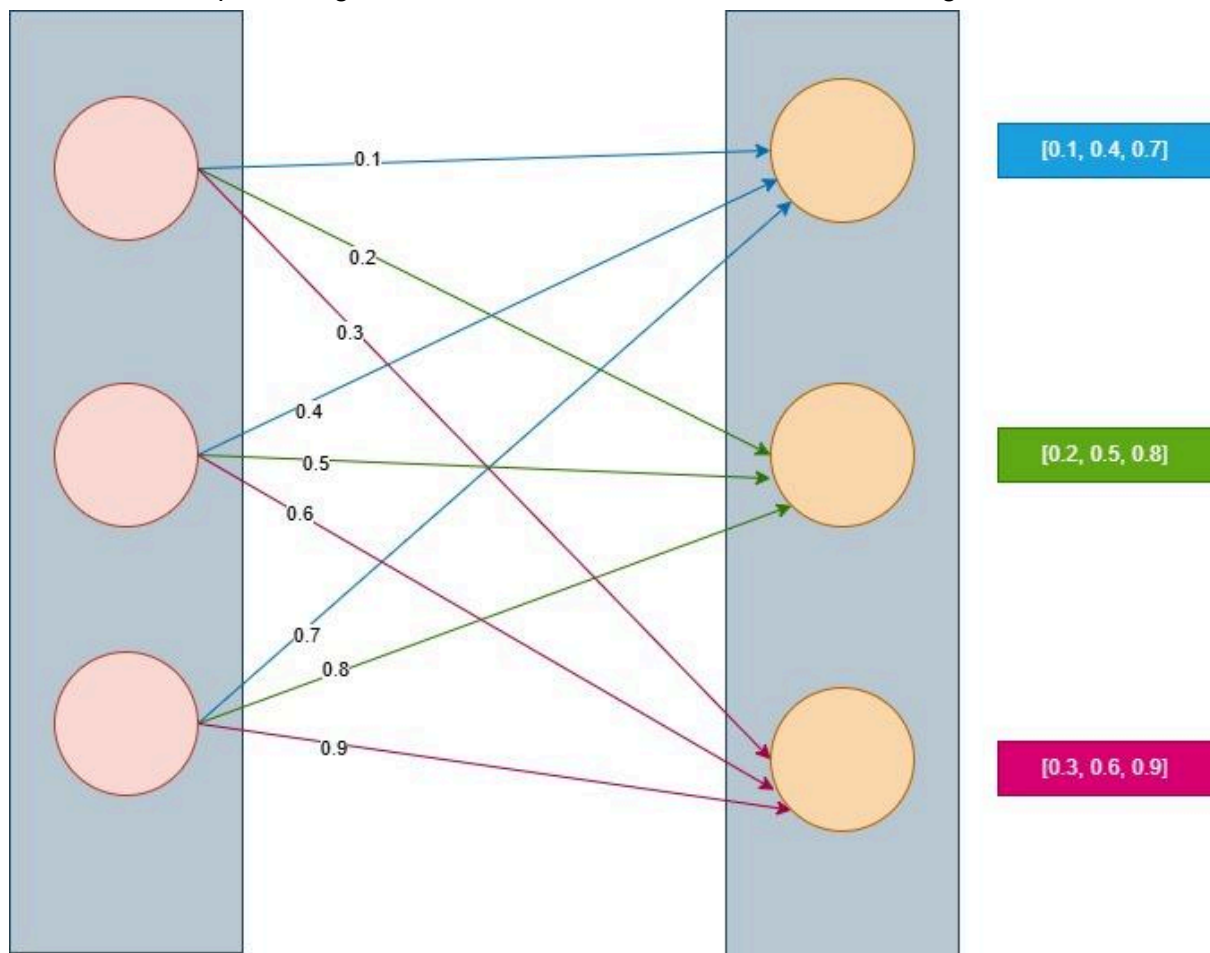
Estructura básica

Para crear la red con backpropagation, definimos primero las estructuras básicas de la red: Neurona, Capa y por último, Red con backpropagation.

La neurona tendrá un vector de pesos, un bias, una salida, y el error de backpropagation (delta). El vector de pesos se inicializará con Xavier para que las activaciones en cada capa no se saturen mucho y el gradiente del backpropagation no se haga muy pequeño.

Tendrá el tamaño de la capa anterior. Tomamos la decisión de no usar listas que digan qué neurona de capa [i] apunta a otra neurona de la capa [i+1] puesto que esto sería muy confuso y poco óptimo a la hora de recorrer cada capa.

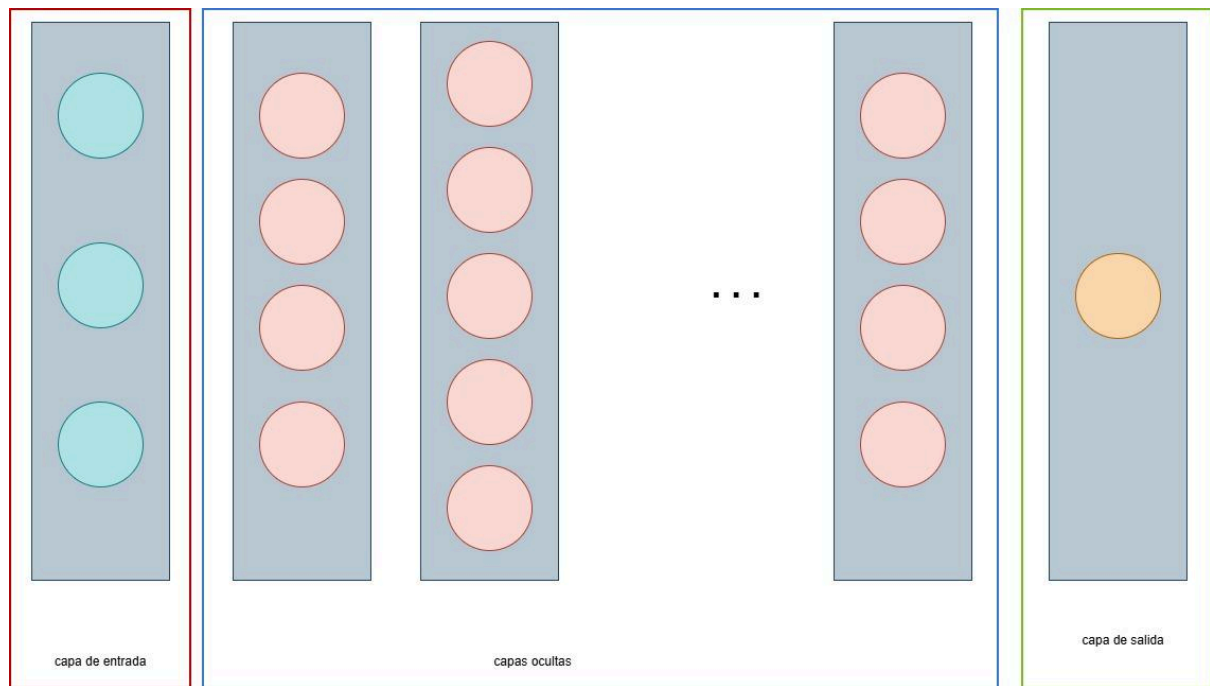
Por ello, nosotros asumimos que cada neurona de la capa anterior apunta a cada neurona de la capa siguiente sí o sí de la siguiente manera.



Con esta imagen se logra comprender que el vector de pesos de cada neurona está formado por los pesos de la capa anterior de forma ordenada.

Por último, delta es la colaboración en el error por parte de la neurona, se calculará en el método backpropagation.

La estructura de Capa contendrá un vector variable de neuronas. Es de esta forma para que, al inicializar la red, podamos poner capas con diferente número de neuronas, tantas como queramos.



Por último, la red está definida como "RedBackPropagation". Incluirá un vector variable de capas, ya que al inicializar la red, tendremos que poner el número de neuronas de la capa de entrada, de un número variable de capas ocultas, y de la capa de salida. También incluirá una tasa de aprendizaje.

Funcionamiento

Inicialización

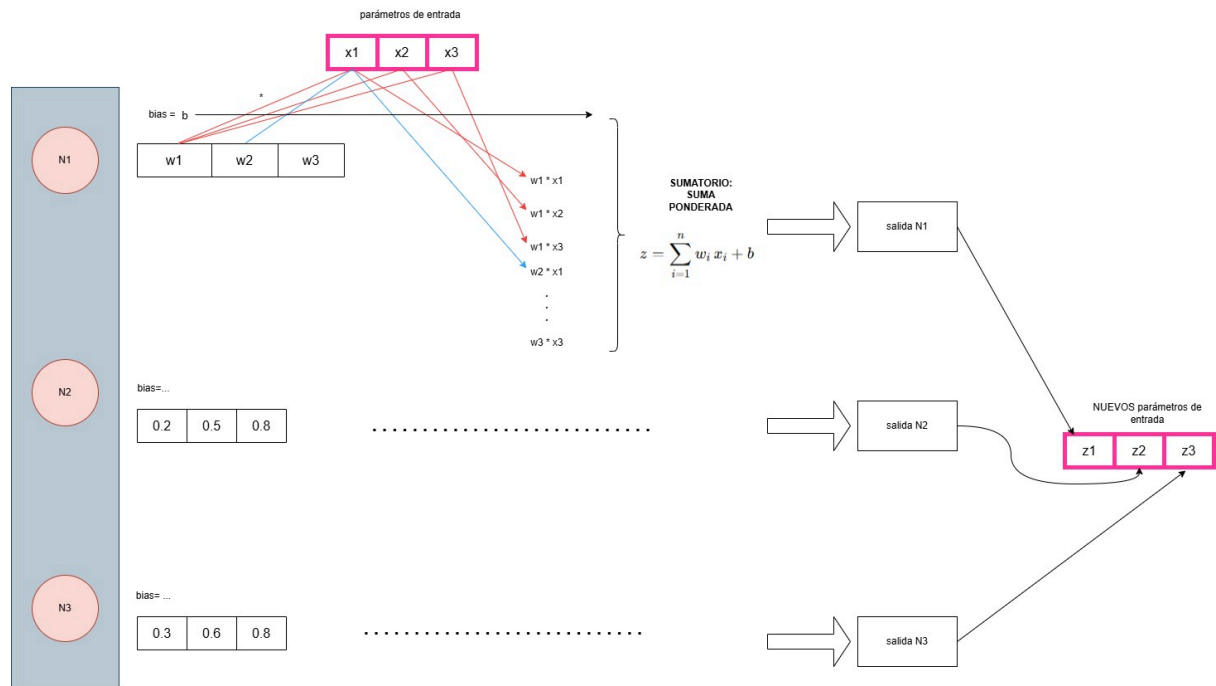
La inicialización de la red se hará en el constructor "RedBackPropagation(const vector<size_t> &arquitectura)". Como las neuronas de entrada no necesitarán pesos ya que se corresponden a los valores de entrada que nosotros le demos, se excluirá la capa de entrada y tomaremos la capa de salida y las capas ocultas. Se debe inicializar la red indicando el número de entradas de la capa de entrada, tantas como parámetros de entrada haya, la cantidad de neuronas de un número variable de capas ocultas, y las neuronas de la capa de salida, correspondientes a la salida esperada.

Por ejemplo: `RedBackPropagation red({11, 20, 20, 20, 1})` define una red con backpropagation con una capa de entrada de 11 neuronas, 3 capas ocultas cada una con 20 neuronas, y una capa de salida con 1 neurona.

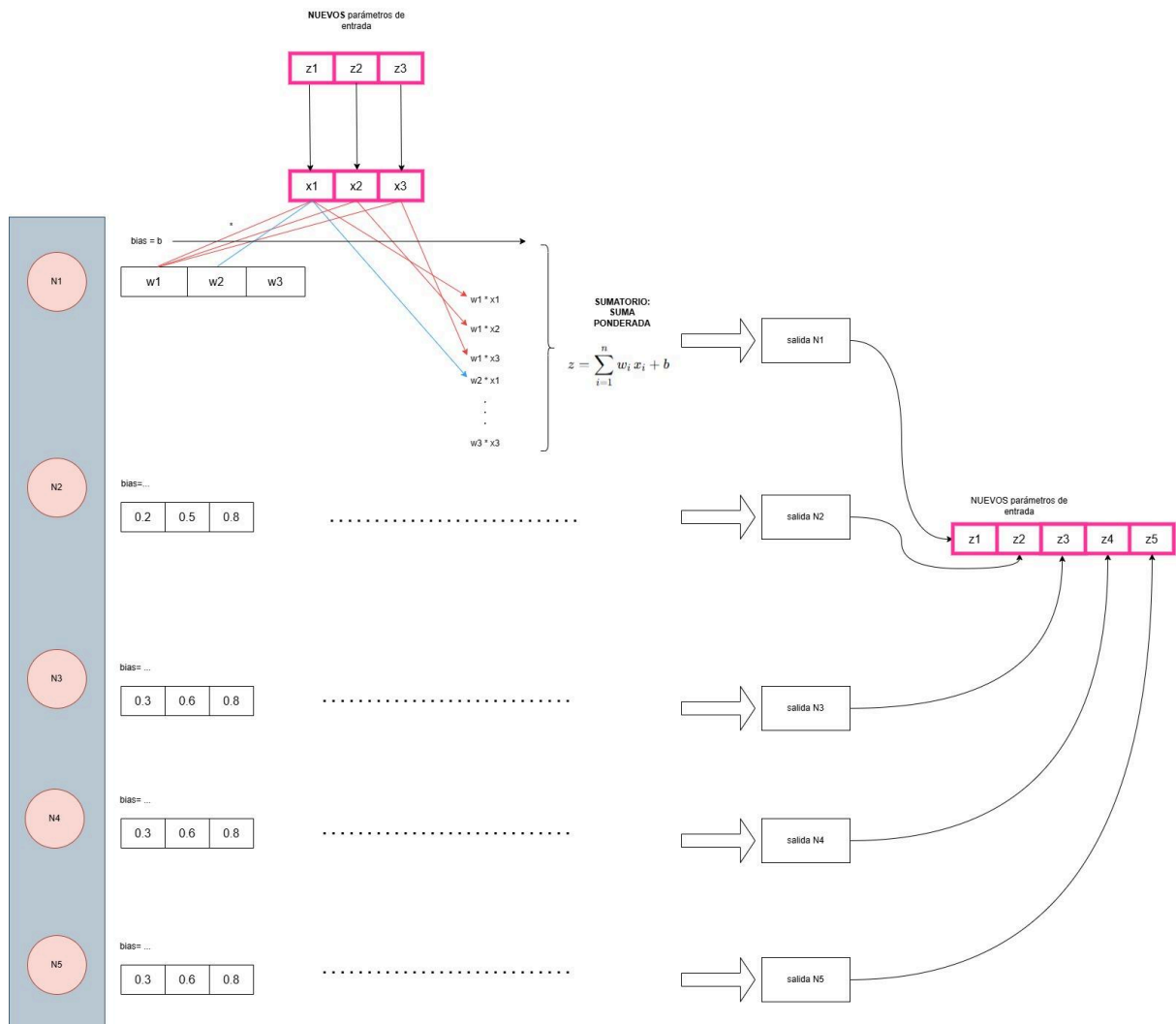
Forward

Para entender las salidas de cada neurona a través de cada capa, pondré un ejemplo muy sencillo: imaginemos que estamos calculando la nueva entrada de la red en una capa de 3 neuronas cuya capa anterior tiene 3 neuronas también. Este ejemplo es sencillo, puesto que partimos del hecho de que, como la capa anterior tiene 3 neuronas, los pesos de la nueva capa serán 3 también, puesto que tienen que coincidir como se declaró anteriormente. Se

calculará la suma ponderada como en la siguiente imagen, y se obtendrá una salida por neurona, a la que, en caso de ser una salida, se dejará tal y como está y como está y si no, se le aplicará una función de activación, como por ejemplo sigmoid:



Pero, ¿y si justo a continuación tenemos una capa de, por ejemplo, 5 neuronas? Como ya sabemos, la capa anterior tiene 3 neuronas y en consecuencia, cada una de las 5 neuronas tendrá un vector de pesos de tamaño 3. Por lo que realizamos los cálculos como antes y obtendremos un vector de salida de tamaño 5:



Si tuviéramos otra capa a continuación con 2 neuronas, como su capa anterior tuvo 5 neuronas, cada neurona de la capa de 2 neuronas tendrá un vector de pesos de tamaño 5, y se cogerá la nueva salida que es la calculada en la imagen, y se realizará la suma ponderada nuevamente.

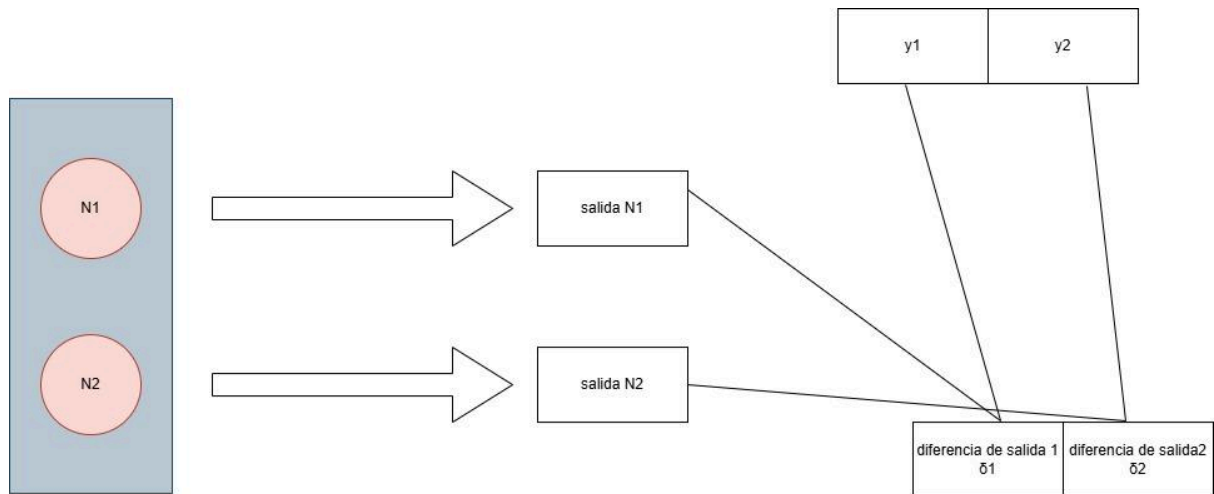
Aunque el error se calcula en backpropagation ya que es en ese método donde tendremos el parámetro de la salida, el error de la red lo obtendremos al final de forwarding, puesto que es el resultado de restar la salida obtenida por la red - la salida real esperada.

A partir de las imágenes conocemos cuales son las nuevas entradas para la red pero, ¿y los pesos de las neuronas? Veremos a continuación cómo se actualizan en **backpropagation**.

Backpropagation

Esta es la parte esencial donde se actualizan los pesos de cada neurona y se calcula cuánto afectó cada neurona al error de la capa anterior.

Lo primero que se hace es calcular el error de la capa de salida, para ello se toman las salidas de cada neurona y se calcula la diferencia respecto del output deseado.



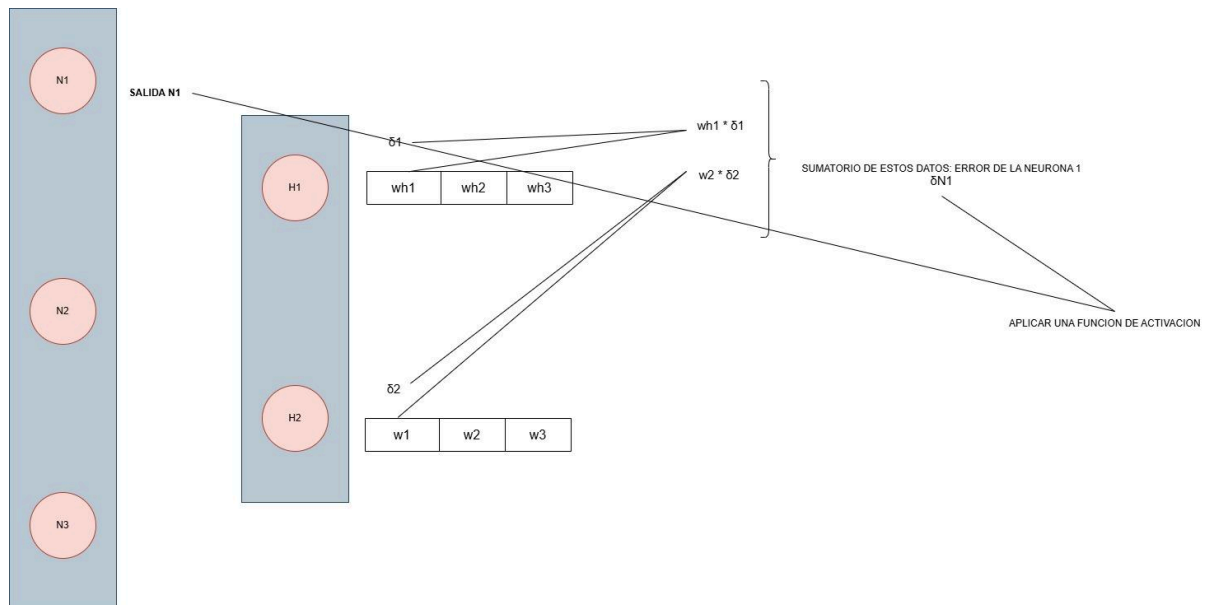
De esta forma conseguimos el delta de las neuronas de la capa de salida. Con este error, calcularemos capa por capa, hacia atrás, ¿cuánto colabora cada neurona de la capa anterior en el error de la capa de salida?, y ¿cuánto colabora cada neurona de la capa [i] en el error de la capa [i+1]?

Como comenté anteriormente, la primera capa no se considerará puesto que tiene la información que nosotros le pasamos. En este apartado, la capa de salida tampoco será considerada durante la vuelta atrás porque ya está incluida en el primer paso del algoritmo descrito en la anterior imagen.

Ahora tenemos el error de cada neurona de la capa de salida. Como sabemos, la capa de salida corresponde a la salida predicha de la red. Por lo que el error δ_1 indica cuánta diferencia hay entre la predicción del parámetro Y1 de la red y el parámetro Y1 real, y lo mismo con Y2.

Observaremos las neuronas de la capa anterior y calcularemos cuánto colaboraron en el error de las neuronas de la capa siguiente, usando la siguiente función.

$$\sum_k w_{ik} \cdot \delta_k$$



De esta forma se calcula el error delta de una neurona de la capa anterior.

Funcionalidad

Para evaluar que la red funciona correctamente, utilizamos nuestro propio dataset sintético, hecho a partir de una función matemática. Este enfoque nos permite tener un control total sobre las relaciones entre las variables de entrada y salida esperada, lo que facilita verificar si la red puede encontrar el patrón subyacente.

Si bien podemos analizar otro conjunto de datos, a simple vista, es muy difícil extraer qué características motivan ciertos resultados.

Como estamos probando una red con backpropagation, tiene más sentido si seleccionamos valores exponenciales en vez de lineales para que no se trate de un problema lineal: creamos un vector de muchos elementos (x, y) y asignamos la salida como $(\exp(x + y), \exp(x - y))$, y el resultado fue el siguiente:

```

INPUT: 0.6, 0.2
Predicción de la red: y1=2.22474, y2=1.4899
Valor real: 2.22554, 1.49182

```

Se ve que incluso con valores no lineales, es capaz de aprender el patrón.

Mejoras implementadas y mejoras posibles

Para favorecer la estabilidad, se cambió la inicialización de pesos simples por la inicialización Xavier. Esto hace que si en una red profunda, si inicializamos los pesos sin cuidado, las activaciones pueden explotar ofreciendo valores enormes, o todo lo contrario, minimizarse, haciendo que todo tienda a 0, por lo que el entrenamiento se hace lento, inestable, o no aprende.

Xavier ajusta la varianza de los pesos según la arquitectura.

Esto se implementó como una solución ya que al probar la función de activación Tanh con el tercer dataset, encontramos errores de aprendizaje, por lo que posiblemente, los gradientes estaban explotando porque 0.5 era muy grande para Tanh en capas ocultas pequeñas.

Además para mayor simplicidad en las pruebas, se ha parametrizado no solo el número de capas y neuronas de cada capa, la tasa de aprendizaje, los datos y el número de épocas, sino también la función de activación.

Una mejora que podríamos haber implementado podría haber sido la regularización con L2.

Datasets para red

Indicador de salud según uso de redes

El dataset que seleccionamos para probar esta red se puede encontrar en el siguiente enlace: <https://www.kaggle.com/datasets/sonalshinde123/social-media-mental-health-indicators-dataset>.

Este conjunto de datos agrupa información como el nombre, edad, género, plataforma, horas de sueño y parámetros relacionados al uso de redes sociales de 5000 personas.

Si se observa a simple vista, resulta curioso cómo hay una relación directa entre la edad, rutina de redes y estado mental en las horas de sueño del individuo. Sin mucha información podemos asumir que las personas de 18 a 25 años duermen menos por coincidir en la etapa del estudiante universitario y el auge de las redes sociales. Mientras que los participantes más mayores de entre 40 y 60 años suelen dormir más, posiblemente por encontrarse en una etapa más estable de la vida y por un uso más controlado de la tecnología. Pero estas son solo conjeturas todavía, las cuales probaremos o refutaremos.

Para entrenar la red, primeramente recogimos todos los datos relevantes, excluyendo el nombre y la fecha en la que se obtuvieron los datos para mantener la privacidad de los usuarios.

▲ mental_state ≡	
Target label: Healthy, At_Risk, or Stressed	
Stressed	92%
Healthy	7%
Other (58)	1%

Además, pese a que el dataset incluye una columna indicativa el estado mental, no se incluyó entre los datos relevantes, puesto que la mayoría (92%), marcaron “Stressed”, y no parece haber una correlación tan fuerte entre estos datos como los hay con otros como el estado de humor, nivel de estrés o nivel de ansiedad.

Como ya sabemos, para entrenar una red necesitaremos un input X y un output Y. Los datos que tomaremos como entrada y salida de la red serán los siguientes:

Entrada X (11 parámetros)	Salida Y (1 parámetro)
- edad - género (0: mujer, 1: hombre, 2: otro) - plataforma (0: Facebook, 1: TikTok, 2: YouTube, 3: WhatsApp, 4: Snapchat) - tiempo en pantalla en minutos - tiempo en redes sociales en minutos - interacciones positivas - interacciones negativas - minutos de actividad física - nivel de ansiedad (1-4) - nivel de estrés (5-9) - nivel de estado de humor(4-7)	- horas de sueño

Para poder evaluar el desempeño de la red correctamente, es necesario dividir la información en datos de entrenamiento: X_train, Y_train, y datos de validación: X_val, Y_val. De esta forma, entrenamos la red solo con los datos de entrenamiento y validamos con datos de validación, lo que permite que el modelo generalice correctamente y no memorice datos de validación.

La capa de entrada tendrá 11 neuronas, correspondientes al número de parámetros de entrada. Como se espera que la red aprenda los patrones para predecir las horas de sueño del individuo, la capa de salida tendrá solo una neurona, correspondiente a las horas de sueño.

Primero, probaremos con una red con 1 sola capa oculta de 10 neuronas, la red se inicializará como RedBackPropagation red_suenyo({11, 10, 1}). Además, el entrenamiento se realizará en 5000 épocas.

Para el entrenamiento, seleccionamos 77% del dataset original como datos de entrenamiento y el 23% restante como datos de validación. De esta forma, podremos ver el rendimiento real de la red a la hora de clasificar estilos de vida de varias personas que nunca observó previamente durante el entrenamiento.

Para medir la precisión del entrenamiento utilizamos MSE (Mean square error)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Que mide qué tan lejos están las predicciones de los valores reales y lo eleva al cuadrado para penalizar los errores más fuertes. Mientras más bajo sea el mse, mejor para la red, indicando que las predicciones están más cerca de los valores reales.

Se calculará el MSE durante el entrenamiento y sobre la validación.

MSE TRAIN	MSE VAL	INTERPRETACIÓN
baja	baja	la red aprende bien
baja	baja y luego sube	precaución: overfitting
baja	sube	overfitting
baja	se queda igual	validación estable NO OVERFITTING

```

OBTENIDO 0.0709372 FRENTE A 0.071
OBTENIDO 0.0671417 FRENTE A 0.067
OBTENIDO 0.0666664 FRENTE A 0.067
OBTENIDO 0.0724354 FRENTE A 0.073
OBTENIDO 0.0720235 FRENTE A 0.072
OBTENIDO 0.0774671 FRENTE A 0.078
OBTENIDO 0.0766783 FRENTE A 0.077
OBTENIDO 0.0698729 FRENTE A 0.07
OBTENIDO 0.0735305 FRENTE A 0.074
OBTENIDO 0.0728143 FRENTE A 0.073
OBTENIDO 0.0660774 FRENTE A 0.066

```

Ejemplo real de varias predicciones de horas de sueño y las horas reales de los individuos de validación

Después de entrenar la red con los datos de entrenamiento y obtener una predicción para los individuos de validación, toca medir el error usando la función del error porcentual absoluto medio MAPE, esto solo se usará para medir el porcentaje de acierto de las predicciones respecto a los outputs reales.

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

Se usará una red con ReLu, con las capas usadas y la selección de datos de entrenamiento y validación siguientes:

```

RedBackPropagation red_suenyo({11, 10, 1});
red_suenyo.tasa_aprendizaje = 0.1;
red_suenyo.entrenar(X_train, Y_train, 5000);

```

```

MAPE: 0.354851 precisión: 99.6451
MSE de entrenamiento: 9.8403e-08
MSE de validacion: 0.000100603

```

Si ponemos ejemplos exagerados para 2 individuos del género masculino y de 24 años diferenciándose uno del otro porque uno de ellos tiene muy buenos hábitos en redes, interacciones mayoritariamente positivas y una salud mental excelente, y el otro pasa un tiempo excesivo en redes, tiene más interacciones negativas y una pésima salud mental, obtenemos los siguientes resultados:

```

Sueño de una persona con buenos habitos de vida y en redes: 7.51524 horas de sueño
Sueño de una persona con malos habitos de vida y en redes: 5.61331 horas de sueño

```

Podemos ver que la persona con buenos hábitos duerme muchísimas más horas mientras que el otro individuo duerme mucho menos que las horas recomendadas de sueño para un adulto de 24 años.

Pruebas y ajustes de hiperparámetros

Ya que se ha probado la red con los parámetros anteriores, veré si experimentando con parámetros puedo disminuir el MSE conseguida anteriormente.

Comenzaré probando con 3 capas ocultas: una de 10 neuronas, otra de 5 y otra de 3:

```
RedBackPropagation red_suenyo({11, 15, 10, 5, 1});
```

El MSE de entrenamiento bajó ligeramente pero el de validación subió. Siguen siendo valores muy pequeños, pero por ese aumento, nos quedaremos con la configuración anterior.

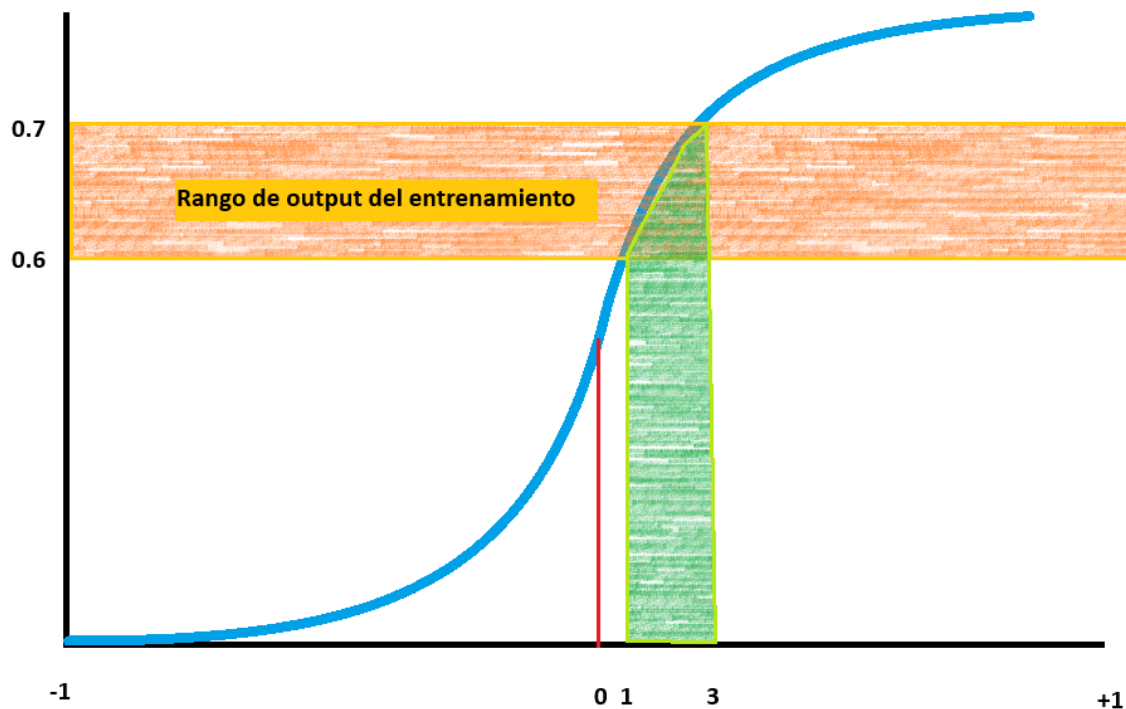
```
MAPE: 0.376902 precisión: 99.6231
MSE de entrenamiento: 9.36175e-08
MSE de validacion: 0.000116025
```

Probaremos con la función de activación Tanh en vez de ReLu:

```
MAPE: 0.39845 precisión: 99.6015
MSE de entrenamiento: 1.59768e-07
MSE de validacion: 0.000138158

// hacia delante
Sueño de una persona con buenos habitos de vida y en redes: 2.40706 horas de sueño
Sueño de una persona con malos habitos de vida y en redes: 5.78079 horas de sueño
```

Como se ve, el porcentaje de predicción es alto, pero si se exageran resultados, informa que una persona sana ha dormido muy poco, lo cual no es lógico. Esto puede suceder porque en ReLu, los valores se sitúan entre $[0, +\infty)$, y si la red predice 0.7, al normalizar = 7 horas. Pero en tanh, los valores van de $[-1, 1]$, por lo que, como la mayoría de los datos de entrenamiento están entre 0.6 y 0.7, la red aprende a mapear la parte pequeña positiva de la curva de tanh.



Por lo que Tanh no es apropiada para este caso.

Por último probamos con Sigmoid, que descartamos puesto que aumenta el valor MSE ambos entrenamiento y validación.

```
MAPE: 0.469232 precisión: 99.5308
MSE de entrenamiento: 1.18963e-07
MSE de validación: 0.000190315

// GRADIENTE DESCENDENTE
Sueño de una persona con buenos hábitos de vida y en redes: 10.097 horas de sueño
Sueño de una persona con malos hábitos de vida y en redes: 5.21211 horas de sueño
```

Para este dataset, la configuración ideal es:

```
RedBackPropagation red_suenyo({11, 10, 1});
red_suenyo.tasa_aprendizaje = 0.1;
red_suenyo.entrenar(X_train, Y_train, 5000);
```

- con activación ReLu.

Tipos de flores iris

El siguiente dataset se trata de la mítica clasificación en 3 tipos de flores iris observando sus características físicas. El dataset se puede encontrar en el siguiente enlace: [Iris Species](#)

Obtener un buen resultado para este dataset será sencillo, puesto que, además del conocimiento base, también usaremos recomendaciones de otros usuarios en la red que hayan experimentado con este dataset.

Para empezar, tomaremos los datos relevantes y los dividiremos en input y output. Este dataset se diferencia del anterior en que el output será una de las posibles clasificaciones

de iris: Iris-setosa, Iris-versicolor, Iris-virginica. Como hay 3 posibilidades, utilizaremos una codificación one-hot para cada una:

Iris-setosa: [1, 0, 0]

Iris-versicolor: [0, 1, 0]

Iris-virginica: [0, 0, 1]

Esto simboliza la posibilidad de que una flor pertenezca a una característica o a otra. Observaremos que a la hora de predecir, cada columna tendrá valores de posibilidad diferente, puesto que el modelo puede decidir la probabilidad de pertenencia de la flor a todas las clases. Por ejemplo, si tenemos que una flor tiene las probabilidades de salida [0.9, 0.07, 0.03], significa que el modelo ha decidido que hay un 90% de posibilidad de que la flor pertenezca a la clase Iris setosa, un 7% de que sea una iris versicolor y un 3% de que sea una iris virginica. Como es evidente, la clase final de esa flor será la columna correspondiente al mayor porcentaje.

El dataset está excelentemente balanceado, puesto que hay el mismo número de muestras para cada clase. El único inconveniente es su orden. Cuando elegimos el porcentaje de datos de entrenamiento, lo que realmente estamos haciendo es calculando el número de ejemplos y dividiendo el vector desde el inicio hasta ese número, por lo que no hay una mezcla explícita en esta parte. Es por ello que, si por ejemplo tenemos 12 ejemplos con las clases ordenadas: 4 de A, 4 de B y 4 de C y seleccionamos desde el primero hasta el séptimo como datos de entrenamiento, a la hora de realizar la validación, no podremos observar el rendimiento real de clasificar otro dato que no sea el de la última clase.

A	A	A	A	B	B	B	B	C	C	C	C
entrenamiento									validación		

Solo se está validando la última clase: no se ve el rendimiento real del modelo.

Para solucionar este problema introducimos el shuffle: mezclamos las flores del vector justo después de leerlas del archivo. De esta forma seguiremos teniendo datos de entrenamiento balanceados y podremos probar todas las categorías:

A	C	B	A	B	C	A	B	C	A	B	C
entrenamiento									validación		

Al realizar shuffle NO ARBITRARIO sobre los datos habrá una mezcla real que será la misma cada vez que compilamos.

De esta forma solucionamos el problema de la selección desbalanceada de los datos.

Los datos de entrenamiento representarán un 70% del dataset mientras que los de validación un 30%.

Después extraemos la matriz de confusión cuya diagonal revela el número de datos bien clasificados por cada clase:

Para calcular MAPE, debemos tomar la clase que predijo con mayoría y compararla al resultado real

$$MAPE = \frac{1}{N} \sum_{i=0}^{N-1} |1 - \hat{y}_{i, \text{clase real}}| \times 100$$

fórmula de MAPE para clasificador one-hot

Si tenemos:

PRED35: 0.98388,-0.0266878,0.0428078 y el valor de salida correcto es 1,0,0, se toma el primer elemento (0.98388) y se compara qué tan cerca lo adivinó mediante la fórmula.

MSE está pensado para regresión, no para una clasificación multiclase ya que no optimiza las posibilidades de la clase y no penaliza bien la confusión entre clases. Por ello, incluiremos las métricas pero no las tendremos muy en cuenta a la hora de realizar el ajuste de hiperparámetros.

La precisión será simplemente el número de predicciones correctas / número total de predicciones * 100.

Como se trata de una clasificación multiclase, usaremos la métrica de recall, que mide el porcentaje de precisión al predecir n muestras de una clase.

La inicialización será la siguiente:

```
RedBackPropagation red_iris({4, 6,, 3});
red_iris.tasa_aprendizaje = 0.01;
double mse = red_iris.entrenar(X_train, Y_train, 10000);
```

- con la activación ReLu

Y su resultado:

```
Recall clase 0: 100%
Recall clase 1: 94.1176%
Recall clase 2: 100%
MAPE: 18.2934 precisión: 97.7778%
MSE de entrenamiento: 0.0802472
MSE de validacion: 0.0388457
```

MATRIZ DE CONFUSION DE CLASIFICACIÓN DE IRIS

CLASE 0	16	0	0
CLASE 1	0	16	1
CLASE 2	0	0	12
	CLASE 0	CLASE 1	CLASE 2 (clase ficticia)

ÍNDICE

CLASE 0: Iris-setosa
CLASE 1: Iris-versicolor
CLASE 2: Iris-virginica

Observamos que la clase 0 y 2 se predicen de forma perfecta, y la clase 1 predice 1 flor como clase 2 erróneamente. Aún así, la diagonal es superior respecto al resto de valores adyacentes, lo que indica que la red predice bien en la mayoría de los casos.

Pruebas y ajustes de hiperparámetros

La red fue entrenada con 1 capa oculta de 6 neuronas y 10000 épocas:

```
RedBackPropagation red_iris({4, 6, 5, 4, 3});  
red_iris.tasa_aprendizaje = 0.01;  
double mse = red_iris.entrenar(X_train, Y_train, 10000);
```

Con esta red se obtuvo un 97.77%.

Si la entrenamos con más capas ocultas como por ejemplo:

```
// ENTRENAMIENTO  
RedBackPropagation red_iris({4, 6, 5, 4, 3});
```

Obtenemos que el recall de la clase 1 aumentó haciendo que todas las muestras de la clase 1 se predigan bien, pero el de la clase 2 disminuyó, por lo que nos quedaremos con solo una capa oculta de 6 neuronas como en el apartado anterior.

```
Recall clase 0: 100%  
Recall clase 1: 100%  
Recall clase 2: 83.3333%  
MAPE: 5.11056 precisión: 95.5556%  
MSE de entrenamiento: 0.0351907  
MSE de validacion: 0.0160752
```

Ahora probaremos la activación Sigmoid:

```
Recall clase 0: 100%  
Recall clase 1: 94.1176%  
Recall clase 2: 83.3333%  
MAPE: 15.6498 precisión: 93.3333%  
MSE de entrenamiento: 0.116598  
MSE de validacion: 0.0349283
```



```
MATRIZ DE CONFUSION DE CLASIFICACIÓN DE IRIS  
CLASE 0    16      0      0  
CLASE 1     0     16      1  
CLASE 2     0      2     10  
          CLASE 0    CLASE 1    CLASE 2    (clase ficticia)
```



```
ÍNDICE  
CLASE 0: Iris-setosa  
CLASE 1: Iris-versicolor  
CLASE 2: Iris-virginica
```

Se ve que empeora la clasificación en la clase 1 y 2.

Si probamos con tanh:


```

Recall clase 0: 100%
Recall clase 1: 100%
Recall clase 2: 50%
MAPE: 25.2295 precisión: 86.6667%
MSE de entrenamiento: 0.213449
MSE de validacion: 0.0780109

MATRIZ DE CONFUSION DE CLASIFICACIÓN DE IRIS
CLASE 0    16      0      0
CLASE 1     0     17      0
CLASE 2     0      6      6
          CLASE 0    CLASE 1    CLASE 2    (clase ficticia)

ÍNDICE
CLASE 0: Iris-setosa
CLASE 1: Iris-versicolor
CLASE 2: Iris-virginica

```

Vemos que la clasificación empeora, por lo que nos quedaremos con la configuración inicial

Heart disease

El dataset se puede encontrar en el siguiente enlace:

<https://www.kaggle.com/datasets/neurocipher/heartdisease>

Se tratará de predecir si un individuo padecerá de alguna enfermedad al corazón. Este también será un problema de clasificación, siendo Presence [1,0] y absence [0,1].

Disponemos de 13 entradas y las dos salidas mencionadas correspondientes a la clasificación

La red se inicializará con los siguientes parámetros:

```

RedBackPropagation red_heart({13, 10, 2});
red_heart.tasa_aprendizaje = 0.01;
double mse = red_heart.entrenar(X_train, Y_train, 10000, activ);

```

y con activación Sigmoid.

Pruebas y ajustes de hiperparámetros

Partimos de la siguiente base:

```

Recall clase 0: 91.4894%
Recall clase 1: 73.5294%
MAPE: 28.4377 precisión: 83.9506%
MSE de entrenamiento: 0.226547
MSE de validacion: 0.133951

MATRIZ DE CONFUSION DE ENFERMEDAD DEL CORAZÓN
CLASE 0  43  4
CLASE 1  9  25
CLASE:0  CLASE 1  (clase ficticia)

ÍNDICE
CLASE 0: Absence
CLASE 1: Presence

```

Se ve como obtuvimos una precisión del 83.95%, jugaremos con las capas y neuronas a ver si podemos aumentarlo:

```
RedBackPropagation red_heart({13, 10, 11, 5, 2});
```

El resultado es peor que el anterior ya que, si bien puede detectar la presencia casi en la misma cantidad, tiene mucho más fallo al decir quienes no tendrán enfermedad del corazón. Se puede ver que predice erróneamente que un individuo tendrá enfermedad del corazón cuando no es así. Por lo que nos quedaremos con las capas iniciales.

```

Recall clase 0: 70.2128%
Recall clase 1: 88.2353%
MAPE: 28.4506 precisión: 77.7778%
MSE de entrenamiento: 0.221674
MSE de validacion: 0.158159

MATRIZ DE CONFUSION DE ENFERMEDAD DEL CORAZÓN
CLASE 0  33  14
CLASE 1  4  30
CLASE:0  CLASE 1  (clase ficticia)

ÍNDICE
CLASE 0: Absence
CLASE 1: Presence

```


Si utilizamos la función de activación ReLu la precisión empeora ligeramente respecto a sigmoid:

```
Recall clase 0: 76.5957%
Recall clase 1: 82.3529%
MAPE: 30.964 precisión: 79.0123%
MSE de entrenamiento: 0.221114
MSE de validacion: 0.152282

MATRIZ DE CONFUSION DE ENFERMEDAD DEL CORAZÓN
CLASE 0 36 11
CLASE 1 6 28
CLASE:0 CLASE 1 (clase ficticia)

ÍNDICE
CLASE 0: Absence
CLASE 1: Presence
```

por lo que probaremos con tanh:

```
Recall clase 0: 85.1064%
Recall clase 1: 85.2941%
MAPE: 30.2775 precisión: 85.1852%
MSE de entrenamiento: 0.212071
MSE de validacion: 0.160707

MATRIZ DE CONFUSION DE ENFERMEDAD DEL CORAZÓN
CLASE 0 40 7
CLASE 1 5 29
CLASE:0 CLASE 1 (clase ficticia)

ÍNDICE
CLASE 0: Absence
CLASE 1: Presence
```

Este fue el que mejor resultado dió, por lo que nos quedaremos con esta característica. Tanh puede haber dado mejores resultados porque, al estar centrada en 0, acelera la convergencia del gradiente y evita que las capas posteriores se saturen rápido.

La configuración que mejores resultados arrojó fue la siguiente:

```
RedBackPropagation red_heart({13, 10, 2});
red_heart.tasa_aprendizaje = 0.01;
double mse = red_heart.entrenar(X_train, Y_train, 10000, activ);
```

- con tanh

Conclusiones

Se puede ver que la red con backpropagation no solo arroja resultados prometedores para problemas de regresión como el de las horas de sueño del usuario, sino también para problemas clásicos de clasificación como el del Iris o enfermedad del corazón.

Aún así el entrenamiento que mayor precisión nos otorga, es el de las horas de sueño, mientras que los resultados con el clasificador no son tan altos en términos de porcentaje de aciertos. Esto puede ser debido a que el clasificador pierde información que la regresión sí usa.

Por ejemplo, si en un clasificador binario se predice (0.55, 0.45), nosotros seleccionamos el argumento máximo 0.55 y asumimos que predice la primera clase, sin tener en cuenta la otra probabilidad. Eso es básicamente como tirar una moneda ya que ambas probabilidades son muy similares.

Red Neuronal con pesos entrenados con Algoritmos Genéticos

Idea básica

La estructura básica de esta red neuronal no varía demasiado de la anterior, se diferencian en la forma que tienen de aprender y mejorar, mientras que la anterior utilizaba backpropagation para comprobar cuánto se había equivocado y transmitir esa información a todas las neuronas, con el genético sin embargo tratamos a las redes neuronales como individuos de una población a la que ponemos a prueba, y mediante una selección por torneo elegimos a las que tienen mejor resultados y las copiaremos con mutaciones añadidas.

Implementación

Como ya he comentado en el apartado anterior, esta red no utiliza backpropagation para aprender seguimos los siguientes pasos:

1. Creamos un vector de redes neuronales todas con la misma arquitectura, cada una tendrá unos pesos aleatorios.

2. A partir de un dataset, probamos parte del dataset para obtener la precisión que obtienen cada una de ellas, y las ordenamos de mayor a menor teniendo la más precisa como primer elemento del vector.
3. Elegimos el top 5% de los individuos y los mutamos reemplazando al resto de individuos y volvemos al paso 2.

Mejoras de la implementación

Función fitness

En un principio la primera función fitness que se usaba era la más simple posible, únicamente se incrementa si acierta la predicción y sino lo hace no. El problema de esta función es que en una red neuronal como la evolutiva, que hace cambios de manera aleatoria no había forma de guiarla hacia la buena dirección. Es decir, si tenemos dos clases para la salida, por ejemplo en el dataset ya mencionado de problemas del corazón, Presencia y Ausencia, y los codificamos con 1 para Presencia y -1 para Ausencia, sumamos uno directamente si la predicción es mayor que 0 si la salida esperada es 1 y si la predicción es menor que 0 si la salida esperada es -1, si nuestra red predice 0.1 cuando la salida esperada es Ausencia le da la misma puntuación que si la red predice 10, cuando la primera red está más cerca del valor real, entonces he considerado sumar fitness proporcionales a la distancia a la que se encuentran, es decir, he decidido dar la puntuación completa, uno, si supera o iguala a 1 o -1 y es correcta, si está entre 1 y 0.5 o -0.5 y -1, se le suma 0,75 si es correcta y si es positiva y correcta 0.5, también he añadido una penalización de 0.25 al fitness si se equivoca al adivinar. Con estos parámetros la red pasa de obtener:

- Con fitness discreto

Arquitectura red: 13, 7, 3, 1

Total: 54

Presence guessed: 0 Presence missed: 24

Absence guessed: 17 Absence missed: 13

Guessed(%): 31.4815%

- Con fitness proporcional

Arquitectura red: 13, 7, 3, 1

Total: 54

Presence guessed: 19 Presence missed: 5

Absence guessed: 27 Absence missed: 3

Guessed(%): 85.1852%

Podemos observar la mejora ya no solo en el porcentaje de aciertos, también en la matriz de confusión con el primer fitness parece que nunca aprende a detectar cuando hay presencia.

Mutaciones

Ahora voy a explicar el sistema de mutación de la red, que para decirlo claro es variar tanto los pesos de la red como el del bias un valor aleatorio. Este valor aleatorio viene de una distribución uniforme aleatoria y la que finalmente usamos depende del grado de mutación que le indicamos por parámetro, tal y como ha quedado la red hay 4 grados de mutaciones,

de menos a más fuerte. Siendo las del último grado la más agresiva, para esos momentos en los que nos vemos atrapados en un mínimo local.

Probando la red con distintos datasets

Heart Disease Prediction

Este dataset tiene como objetivo predecir si una persona padece de cardiopatía o no, a partir de unos parámetros que veremos a continuación, cada fila del dataset referencia a un paciente, es un dataset para clasificadores binarios, ya que solo tiene que aprender presencia o ausencia.

Las entradas del dataset son:

- **Age:** Edad del paciente
- **Sex:** Género del paciente, 1 Hombre, 0 femenino
- **Chest Pain Type:** Tipo de dolor en el pecho: este valor está discretizado en cuatro valores dependiendo del grado
 - 1 = Typical angina
 - 2 = Atypical angina
 - 3 = Non-anginal pain
 - 4 = Asymptomatic
- **BP:** La presión arterial
- **Cholesterol:** Colesterol
- **FBS over 120:** Si la glucosa es mayor a 120 después de 8-10 horas. 0 si no lo es y 1 si lo es.
- **EKG results:** Otra entrada discreta, que describe los resultados obtenidos de un electrocardiograma, {0, 1, 2}
- **Max HR:** El máximo valor de latidos por minuto del paciente
- **Exercise angina:** Angina inducida por deporte, un dolor de pecho ocurrido tras hacer ejercicio, 1 si ha ocurrido, 0 sino
- **ST depression:** Indica si la actividad eléctrica del corazón es irregular, valor flotante
- **Slope of ST:** la pendiente de la línea ST
- **Number of vessels fluoro:** Número de arterias visibles mediante una fluoroscopia
- **Thallium:** Indica como de bien fluye la sangre por el corazón al hacer ejercicio y durante el ejercicio.

Hay que aclarar porque así lo indica el dataset, que este dataset es meramente educativo y no pretende servir de base real para nada. Me falta comentar que hay datos de las entradas que han sido normalizados debido a lo grande que eran con respecto al resto de valores, estas entradas son la edad, el colesterol, la presión arterial y Max HR.

Pruebas y ajustes de hiperparametros

Sin contar los datos de entrenamiento la red neuronal evolutiva tiene dos parámetros que podemos tocar para intentar obtener mejores resultados, uno de ellos es el que tienen todas las redes neuronales que son las epochs o iteraciones, veces sobre las que iteramos sobre los datos de entrenamiento, y el otro valor es el número de la población.

Iniciamos las pruebas de la red con la siguiente arquitectura:

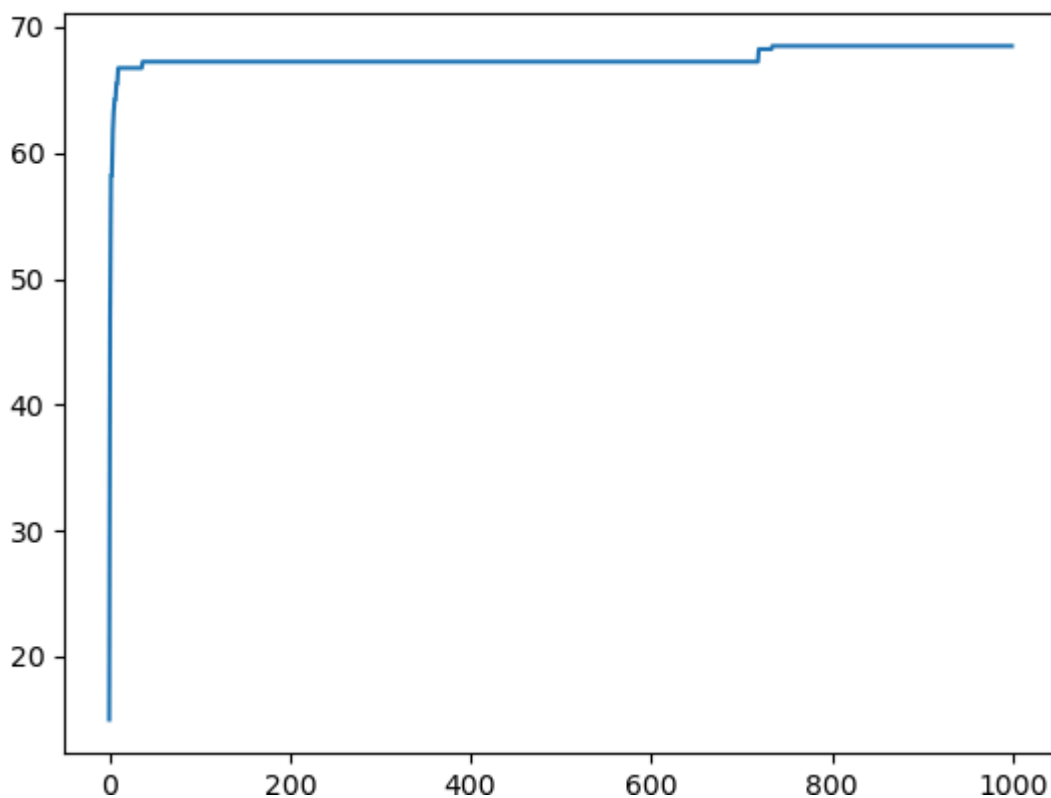
- `{inputs[0].size(), 7, 3, 1}`
- 350 individuos
- 1000 iteraciones

Una capa de 13, seguida de una de 7, otra de 3 y por último la neurona de salida.

Primero tenemos que ver con cuántos datos tratamos y como de repartida está la muestra en las dos clases y es que de 270 datos un 56% dan como resultado Absence. Esta es el resultado

Recall Presence: 70.93 %
 Recall Absence: 82.52 %
 Presence guessed: 61 Presence missed: 25
 Absence guessed: 85 Absence missed: 18
 Guessed(%): 77.25 %

Podemos ver:



La siguiente gráfica muestra como va evolucionando el valor de la función fitness a través de las iteraciones, muestra perfectamente cómo funciona la red neuronal a través del tiempo, podemos observar que hay momentos en los que la función no evoluciona nada y queda igual durante muchas iteraciones hasta que de repente aparece una mutación que hace que la función fitness mejore y de ahí surjan mejoras hasta que vuelve a entrar en un mínimo local. También se puede ver como al principio se mejora mucho más que al final.

Ahora podemos probar a incrementar los individuos a 500, ya que a mayor la población, mayor es la cantidad de individuos con mutaciones fuertes para permitarnos salir de mínimos locales:

Recall Presence: 48.84 %

Recall Absence: 89.32 %
Presence guessed: 42 Presence missed: 44
Absence guessed: 92 Absence missed: 11
Guessed(%): 70.90 %

Podemos observar que aumenta un 7% la precisión para detectar Absence y el porcentaje de Presence ha bajado un 21,2%, parece que incrementar la cantidad de individuos ha hecho que descubran que el dataset tiene una mayoría de Absence y ha aprendido a identificar los Absence más que los Presence, esto también puede ser a que en el 30% de los datos de entrenamiento también hay una mayoría Absence, procedemos a reordenar las filas del dataset para que haya más o menos las mismas de cada clase.

El resultado es el siguiente:

Recall Presence: 74.68 %
Recall Absence: 71.82 %
Presence guessed: 59 Presence missed: 20
Absence guessed: 79 Absence missed: 31
Guessed(%): 73.02 %

Podemos observar como ha mejorado con respecto a la anterior pero aún así no llega a los resultados marcados por 350 individuos, que ahora con esta nueva organización de datos obtienes:

Recall Presence: 65.82 %
Recall Absence: 82.73 %
Presence guessed: 52 Presence missed: 27
Absence guessed: 91 Absence missed: 19
Guessed(%): 75.66%

Vemos como los datos no llegan a ser los mismo que el otro de 350, puede ser debido a que la última vez encontró una mutación que en esta no ha aparecido, cosa que puede pasar en algoritmos como estos, pero aún así el resultado que obtiene es mayor que con 500 individuos.

Ahora que parece que hemos encontrado un buen parámetro para los individuos, me gustaría probar con distintas arquitecturas de redes neuronales, como puede ser:

```
RedBackPropagation red({inputs[0].size(), 10, 1})
```

Probaremos con una solo capa de 10 neuronas:

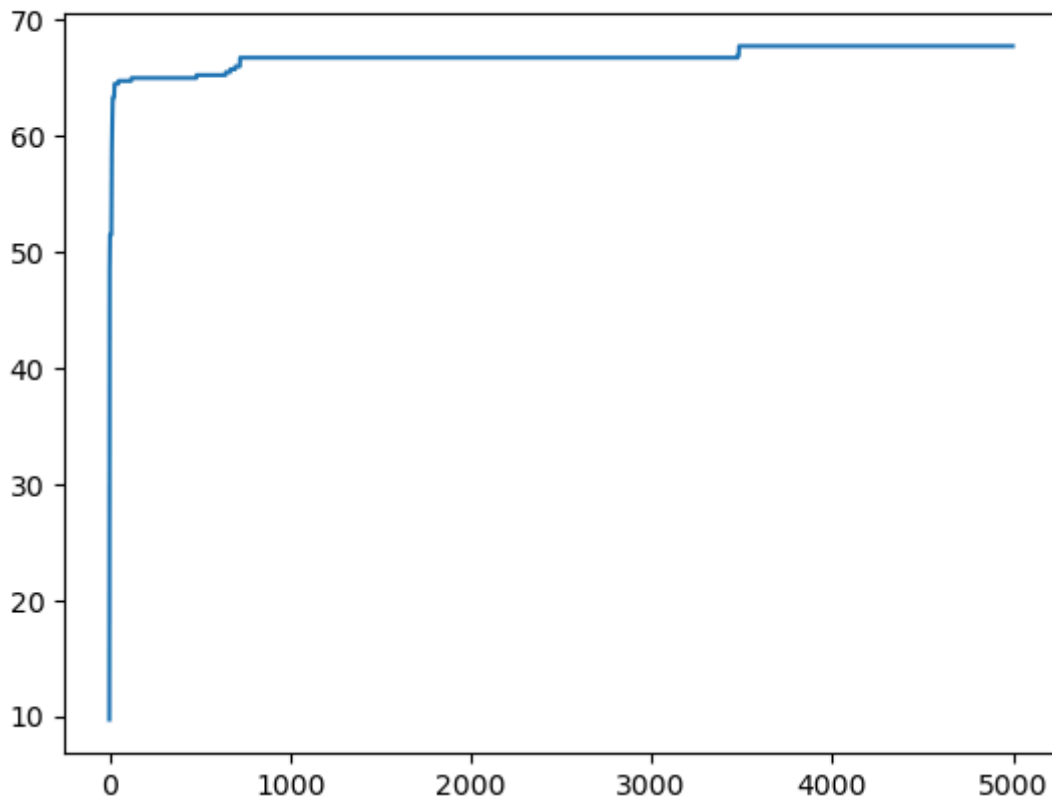
Recall Presence: 83.54 %
Recall Absence: 72.73 %
Presence guessed: 66 Presence missed: 13
Absence guessed: 80 Absence missed: 30
Guessed(%): 77.25%

Obtenemos un buen resultado tal y como el primero obtenido.

Por último vamos hacer una prueba extrema en la que aumentamos la población a unos 500 individuos y 5000 iteraciones, el resultado es el siguiente:

Recall Presence: 64.557%
Recall Absence: 56.3636
Presence guessed: 51 Presence missed: 28
Absence guessed: 62 Absence missed: 48

Guessed(%): 59.7884%



La gráfica nos muestra lo que ha pasado en esta ejecución y es que se ha quedado estancado durante más de mil iteraciones y por tanto no ha podido aprender tanto como el resto. Con esta información debemos ver qué mutaciones son las que más nos hacen mejorar para poder hacerlas más frecuentemente o incluso añadir de alguna forma que tras estar atrapado x iteraciones genere más mutaciones fuertes para intentar salir de ahí y menos de las sutiles. A continuación copio unas filas que indican la iteración del torneo, el número del mejor individuo y el valor de la función fitness:

86 39:65.5

...

133 36:66

...

144 39:67

145 0:67

146 0:67

147 0:67

Podemos ver cómo los cambios son del primer bloque, mutaciones sutiles, por lo que parece que es más suerte que otra cosa, ya que esa es la mutación más común.

Tipos de flores iris

Este dataset ya se ha explicado en apartados anteriores así que me ceñiré a comentar las modificaciones que hacemos al dataset antes de entrenar, la primera es normalizar cada

uno de los datos para tener valores de entre 0 y 1, con unas pequeñas líneas de código podemos averiguar cada uno de los rangos de las 4 entradas que tiene el dataset:

```
Max SLength: 7.9 Min SLength: 4.3 Max SWidth: 4.4 Min SWidth: 2
Max PLength: 6.9 Min PLength: 1 Max PWidth: 2.5 Min PWidth: 0.1
```

Además como ya se ha comentado en el apartado de redes neuronales cabe mezclar las filas, ya que tal y como están en un inicio en el dataset están de forma secuencial y a la hora de entrenar puede llegar a no tener datos de una o más de las tres clases.

La estructura de la red con la que vamos a trabajar, tiene 4 entradas, una capa de 6 neuronas de manera intermedia y tres salidas;

Pruebas y ajustes de hiperparametros

Empezamos las pruebas con 350 individuos, 400 rondas y con la estrategia de entrenamiento 30% y prueba 70%. En una buena prueba, debido a la naturaleza del algoritmo, podemos a llegar a obtener estos resultados:

CLASE 0:	35	0	0
CLASE 1:	0	28	1
CLASE 2:	0	3	30
	CLASE 1	CLASE 2	CLASE 3
	35	35	35

Recall Class1: 100 %
 Recall Class2: 80 %
 Recall Class3: 85.7143 %
 Total accuracy: 88.5714 %

Podemos probar aumentar el número de datos de entrenamiento con un 70, 30 para ver si llega a aumentar los porcentajes, porque en las pruebas hechas con estos parámetros, se llega al 100% de accuracy en la parte de entrenamiento pero no se refleja correctamente en el test. Los resultados que obtenemos son:

CLASE 0:	15	0	0
CLASE 1:	0	14	1
CLASE 2:	0	0	14
	CLASE 1	CLASE 2	CLASE 3
	15	15	15

Recall Class1: 100 %
 Recall Class2: 93.3333 %
 Recall Class3: 93.3333 %
 Total accuracy: 95.5556 %

Podemos observar cómo obtenemos porcentajes máximos, para asegurarnos al 100% de que la red está aprendiendo correctamente vamos a repartir al 50% los datos y ver cual es el resultado:

CLASE 0:	25	0	0
CLASE 1:	0	24	1
CLASE 2:	0	1	23
	CLASE 1	CLASE 2	CLASE 3
	25	25	25

Recall Class1: 100 %

Recall Class2: 96 %
Recall Class3: 92 %
Total accuracy: 96 %

Pues en este caso podemos observar definitivamente como la red neuronal aprende casi a la perfección del dataset obteniendo porcentajes muy cercanos al 100%, lo sorprendente de esto también es la poca carga computacional que ha necesitado comparado con la prueba anterior, esto puede deberse a lo pequeño que es el dataset o a lo bien repartido que está para las tres clase que tiene.

Indicador de salud según uso de redes

Los primeros retoques que hacemos sobre los datos de entrada, son elegir los datos de entrada, en este caso elegiremos los mismo que en las redes neuronales, que son la edad, género, plataforma tiempo pantalla, tempo en redes sociales, interacciones negativas, interacciones positivas, minutos de actividad física, nivel de ansiedad, nivel de estrés y estado de humor. Elementos como plataforma y género debemos formatearlos para asignar a una palabra un valor. Para el género (0: Male, 1:Female, 2:Other) y para las plataformas (0: Instagram, 1: Snapchat, 2: Facebook, 3: WhatsApp, 4: TikTok, 5: YouTube, 6: Twitter). Estos valores, además del resto excepto la salida, los normalizamos para que estén entre 0 y 1. La salida, o valor objetivo, será las horas de sueño que tiene el paciente según todas las entradas.

Pruebas y ajustes de hiperparametros

El dataset tiene 5000 entradas lo cual significa que la fase de entrenamiento va a tener un coste computacional superior al resto de pruebas hechas anteriormente, por estos motivos, haremos únicamente 100 iteraciones, 300 individuos y 1500 entradas para entrenar que es el 30% correspondiente y la arquitectura de la red es la misma que la primera de la red neuronal, es decir, {11, 15, 10, 5, 1}. También hay que comentar qué como en este dataset no se busca clasificar sino predecir un valor, habrá que hacer cambios tanto en la función de test, para comprobar como de bueno es el resultado final, y la función fitness que como hemos comentado en su apartado correspondiente es lo que determina cual de nuestras redes es mejor que el resto. Para la función fitness hemos decidido dar valores proporcionales a como de cerca estaba la predicción a la realidad, dando los puntos completos si entraba en un rango de entre 0.01 y -0.01 la diferencia, luego 0.75 si se encuentra entre 0.1 y -0.1, 0.5 si se encuentra entre 0.5 y -0.5 y -0.25 si la diferencia es mayor. Y para la función test únicamente calculamos y mostramos el error medio de las entradas restantes.

En la primera ejecución realizada, hemos obtenido este resultado, hemos de decir que justo en la iteración 65 se había alcanzado el accuracy completo y que para hacer el entrenamiento el coste temporal ha sido considerable. A pesar de que el resultado no es malo, se puede ver, como la función fitness puede haber tenido que ver en el límite de cuanto se ha podido reducir el coste.

Mean Error: 0.00135384

Error tras la primera prueba

Para la siguiente prueba nuestro accuracy será el error medio obtenido por cada red y por tanto se ordenarán las redes de menor a mayor error medio, lo cual esperamos que dé un mejor resultado.

Mean Error: 0.0121663

Nuevo resultado

Como podemos observar la mejora no ha sido muy notoria, sino todo lo contrario ha empeorado debido a que la puntuación no depende del rango y fuerza la restricción. Por lo que el camino a seguir no es ordenar por el error de media sino reducir el rango en el que damos los valores máximos.

Conclusiones

Para concluir, este apartado sobre la red evolutiva tenemos que ser honestos con todos los aspectos del algoritmo, es decir, el algoritmo depende de mucho de la suerte, es decir, con dos pruebas distintas puede obtener distintos resultados y puede pasar de un 50% a un 80% si se queda atascado en un mínimo local y no consigue salir. Otra desventaja es el coste temporal, hay que tener en cuenta que se hacen las pruebas con el dataset no solo tantas veces como épocas hay, sino que también se le añade el coste de que hay que hacer esto por cada individuo en la red por lo que el coste se multiplica y en muchas ocasiones esto puede hacer que con datasets muy grandes el entrenamiento tarde demasiado, también es un algoritmo muy demandante en el que la función fitness y el grado de las mutaciones marcan el cómo de bien pueden llegar a funcionar. Con todo esto dicho, es cierto que una vez encuentras los parámetros indicados y con algo de suerte se llegan a obtener muy buenos resultados, con porcentajes muy altos, además hay que admitir que este algoritmo se suele utilizar con varias simulaciones en paralelos para calcular sus valores de forma independiente y es ahí donde se ve el verdadero potencial del algoritmo pudiendo aumentar la cantidad de individuos sin afectar de forma tan grave al tiempo, aún así espero haber podido llegar a mostrar un ápice de lo que puede llegar a ser.

Bibliografía

[I finally understand backprop : r/learnmachinelearning](https://www.reddit.com/r/learnmachinelearning/comments/9vurfk/i_finally_understand_backprop/)
<https://youtu.be/nw9vurfkR9c>